TUGAS PERTEMUAN 6

IMAGE PROCESSING

“Fourier Transform”

Disusun oleh:

Muhammad Dawi Syauqi

NIM. 2311532009

Dosen Pengampu:

Afdhal Dinilhak, S.Kom, M.Kom

PROGRAM STUDI S1-INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

A. Resume Materi

Transformasi Fourier adalah salah satu metode transformasi yang sangat penting dalam bidang pemrosesan sinyal (*Signal Processing*), terutama pada ranah pengolahan citra. Secara umum, sebuah sinyal divisualisasikan melalui plot amplitudo terhadap waktu (untuk fungsi satu dimensi) atau terhadap posisi spasial (untuk fungsi dua dimensi). Dalam pengaplikasiannya, memanipulasi sinyal secara langsung di domain waktu atau spasial seringkali rumit, seperti saat menjalankan operasi konvolusi. Namun, jika fungsi tersebut dipindahkan ke ranah frekuensi, proses konvolusi dapat dilakukan dengan lebih sederhana sebagai operasi perkalian biasa.

Transformasi Fourier berfungsi sebagai alat bantu (*tool*) untuk mengubah fungsi dari ranah waktu/spasial menjadi ranah frekuensi. Kemudian untuk mengembalikannya, kita menggunakan Transformasi Fourier Balikan (*Inverse Fourier Transform*). Pada dasarnya, transformasi ini bekerja dengan menguraikan sinyal atau gelombang menjadi sekumpulan gelombang sinusoida dengan ragam frekuensi yang ekuivalen dengan sinyal aslinya.

Dalam pengolahan citra, Transformasi Fourier digunakan untuk menganalisis frekuensi pada operasi, seperti perekaman, perbaikan kualitas, restorasi, hingga pengkodean gambar. Analisis frekuensi ini memungkinkan kita untuk memodifikasi frekuensi pada gambar, yang secara langsung memengaruhi spektrum visualnya mulai dari gambar yang berkontras pudar hingga gambar dengan detail yang tajam. Contohnya, saat proses perekaman, kontras gambar bisa saja memudar karena transisi intensitas yang lambat (hilangnya informasi pada frekuensi tinggi). Untuk mengatasinya dan meningkatkan kualitas visual, kita dapat mengaplikasikan filter frekuensi tinggi (*highpass filter*) agar intensitas pada piksel yang kabur tersebut dapat dipertajam.

1. Continuous Fourier Transform (CTF)

Continuous Fourier Transform (CFT) adalah representasi matematis yang menunjukkan bahwa sebuah fungsi kontinu yang bernilai riil (baik periodik maupun non-periodik asalkan luas di bawah kurvanya terbatas) selalu dapat diekspresikan sebagai penjumlahan dari gelombang sinus dan kosinus pada rentang frekuensi

yang berbeda. Transformasi ini mengonversi sinyal dari domain spasial atau domain waktu menjadi representasi di domain frekuensi, di mana hasil konversi tersebut umumnya mengandung komponen bilangan kompleks. Ciri terpenting dari operasi ini adalah tidak ada informasi yang hilang, fungsi frekuensi tersebut dapat dipulihkan sepenuhnya kembali ke domain asalnya secara presisi menggunakan operasi transformasi kebalikan atau inversnya.

Rumus CFT ID:

$$\boldsymbol{F(\mu) = \int_{-\infty}^{\infty} f(t) e^{-j2\pi\mu t} dt}$$

Rumus Inverse CFT:

$$f(t) = \int_{-\infty}^{\infty} F(\mu) e^{j2\pi\mu t} (d\mu)$$

Keterangan:

a. $F(\mu)$, fungsi di domain frekuensi
b. $f(t)$, fungsi sinyal kontinu pada domain spasial
c. $j = \sqrt{(-1)}$, unit imajiner
d. $e^{j2\pi\mu t}$, sebagai fungsi basis (gelombang kompleks)

2. Descrete Fourier Trasform (DFT) dan Inverse Discrete Fourier Transform

DFT adalah versi diskret dari CFT yang dirancang agar data hasil *sampling* (seperti susunan piksel pada citra digital) dapat diproses oleh komputer. DFT memperlakukan deret sampel data seolah-olah bersifat periodik tak terhingga. Untuk membalikkan matriks data dari domain frekuensi kembali menjadi gambar spasial diskret, digunakan operasi IDFT.

Rumus DFT untuk citra 2D berukuran M x N

$$\boldsymbol{F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(ux/M+vy/N)}}$$

Rumus IDFT untuk citra 2D berukuran M x N

$$\boldsymbol{f(x,y) = \frac{1}{MN}\sum_{u=0}^{M-1}\sum_{v=0}^{N-1} F(u,v)e^{j2\pi(ux/M+vy/N)}}$$

Keterangan:

a. $f(x,y)$, Fungsi matriks citra dua dimensi pada domain spasial
b. x,y, Variable koordinat titik pada sampel diskret
c. M,N, Batas jumlah totoal sampel/ ukuran Panjang dan lebar dimensi citra
d. $F(u,v)$, Representasi mmatriks diskret pada domain frekuensi
e. u,v, Variabel frekuensi diskret

3. Fast Fourier Trasnform (FFT) dan Inverse Fast Fourier Transform (IFFT)

Menghitung DFT dengan cara biasa memakan waktu yang sangat lama dan berat untuk citra beresolusi tinggi. Maka dari itu FFT hadir sebagai algoritma cerdas yang memecah perhitungan tersebut secara bertahap, sehingga jumlah komputasi berkurang drastis menjadi jauh lebih cepat dan ringan. IFFT adalah algoritma serupa untuk menginversi perhitungan tersebut kembali ke domain asalnya dengan tingkat kecepatan yang sama.

DFT standar memiliki kompleksitas komputasi $O(N^2)$ yang sangat lambat. FFT mengurangi kompleksitas menjadi $O(NlogN)$, memberikan percepatan dramatis terutama untuk data besar.

Prinsip Algoritma Cooley-Tukey pada FFT:

Algoritma ini memanfaatkan strategi *divide-and-conquer*, yaitu:

1. Membagi DFT berukuran N menjadi dua bagian DFT berukuran $N/2$ (memisahkan sampel genap dan ganjil).
2. Masing-masing bagian tersebut dipecah lagi menjadi DFT berukuran $N/4$.

3. Pemecahan rekursif ini terus diulang hingga menyisakan ukuran DFT terkecil (ukuran 2) yang mudah dihitung.
4. Terakhir, gabungkan kembali hasil komputasi sub-DFT menggunakan metode *butterfly computation*.

Agar berjalan efisien, ukuran citra idealnya harus berupa bilangan pangkat dua (2^n, misalnya 64, 128, 256, 512, dst). Jika ukurannya tidak pas, teknik penambahan nol (*zero-padding*) bisa digunakan. Terdapat juga *Inverse Fast Fourier Transform* (IFFT) yang merupakan versi efisien dari IDFT dengan kompleksitas serupa. Di dunia pemrograman, pustaka populer seperti OpenCV (cv2.dft), NumPy (numpy.fft), dan MATLAB (fft2) sudah menyediakan fungsi algoritma FFT ini secara bawaan

4. Properti Fourier Transform

a. Translasi (Pergeseran)

$$\boldsymbol{f(x - x_0, y - y_0) \Leftrightarrow F(u, v)e^{-j2\pi(ux_0/M + vy_0/N)}}$$

Sifat ini menandakan bahwa jika sebuah gambar digeser posisinya di dalam domain spasial, hal tersebut tidak akan mengubah besaran nilai amplitudo pada spektrum frekuensinya sama sekali. Penggeseran ini hanya memengaruhi nilai pergeseran fasenya.

b. Rotasi

$$\boldsymbol{f(r, \theta + \theta_0) \Leftrightarrow F(\omega, \varphi + \theta_0)}$$

Jika gambar asli diputar dengan sudut tertentu, sifat ini memastikan bahwa matriks spektrum frekuensinya juga akan ikut berputar mengikuti sudut dan arah yang persis sama.

c. Periodisitas

$$\boldsymbol{F(u, v) = F(u + k_1 M, v + k_2 N)}$$

Khusus pada hasil transformasi diskret, algoritma akan menghasilkan matriks frekuensi yang diasumsikan terus mengulang nilainya sendiri secara periodik, melampaui batas dimensi gambar tersebut.

d. Simetri Konjugat

$$\boldsymbol{F^*(u,v) = F(-u,-v)}$$

Untuk data gambar di dunia nyata (bernilai riil), spektrum frekuensi yang dihasilkan selalu memiliki bentuk pencerminan. Artinya, separuh bagian spektrum merupakan cerminan visual dari separuh bagian lainnya.

e. Teorema Konvolusi

$$\boldsymbol{(f * h)(x,y) \Leftrightarrow (F \cdot H)(u,v)}$$

Teorema ini membuktikan bahwa proses *filtering* (penyaringan) dengan teknik konvolusi matriks yang rumit di domain spasial dapat dihitung secara jauh lebih cepat dan sederhana lewat operasi perkalian biasa di domain frekuensi.

B. Implementasi Fourier Transform dalam Pemrosesan Citra

Implementasi dari *filtering* (pemfilteran) dilakukan dengan mengubah sinyal citra dasar ke domain frekuensi (menggunakan prosedur transformasi seperti FFT yang dilengkapi metode penambahan baris bernilai nol (*zero-padding*) untuk mencegah tabrakan *wraparound error*), kemudian mengalikan spektrum secara *element-wise* dengan filter transfer fungsi $H(u,v)$, dan menghitung Invers Transformasinya agar data tersebut kembali menjadi citra visual.

Penerapan:

- **Pengahalusan / Pengeaburan (Image Smoothing)**: Dilakukan melalui penggunaan *Lowpass Filter*, yaitu mentransmisi komponen frekuensi rendah yang umumnya berkaitan dengan corak intensitas halus seragam, namun dengan kuat melemahkan elemen frekuensi tinggi (misal: struktur tepi, *noise*) untuk mengaburkan (*blur*) gambar. Implementasinya mencakup ideal, *Butterworth*, hingga *Gaussian* di mana model matematika *Gaussian* digunakan bila diinginkan citra blur mulus yang terhindar dari efek kemunculan artefak *ringing* di pinggiran yang umum di *Ideal Lowpass*.
- **Penajaman Gambar (Image Sharpening)**: Melalui instrumen *Highpass Filters*, kebalikan dari pelembutan. Menahan atau menghilangkan frekuensi bernada rendah (ruang mulus dalam gambar) dan membiarkan data berfrekuensi tinggi lewat agar transisi tajam (tepi bangunan/garis potong komponen) menjadi

jauh lebih difokuskan dan ditekankan. Modifikasi matematis semacam diferensiasi Laplacian juga digunakan di ranah Fourier ini.

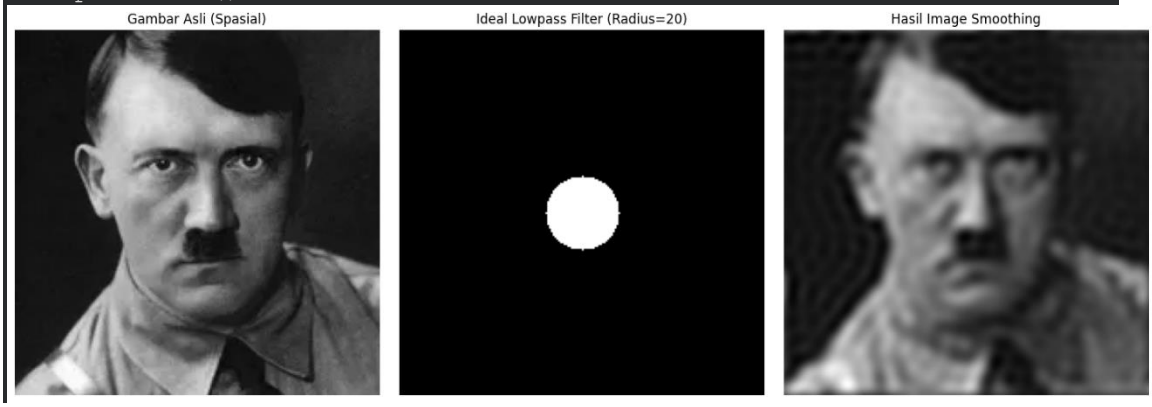
- **Homomorphic Filtering**: Secara simultan menekan frekuensi dari sumber tingkat pencahayaan tidak seragam dalam citra sembari memperkuat komponen gradien *reflectance*, yakni melalui pencarian transformasi Fourier terhadap logaritma alamiah (*natural logarithm*) dari matriks spasial gambarnya.
- **Selective Filtering (Notch/Band Reject)**: Penggunaan *Notch Filters* untuk mendeteksi serta menghalangi sekumpulan impuls titik kecil pada bagian matriks frekuensi domain tertentu yang tak lazim. Fungsi penerapan utamanya seringkali secara interaktif dikonfigurasi guna menghilangkan distorsi derau periodik (*noise* visual terpola), misal artefak *moiré* di proses *scan* foto cetak atau interferensi sinyal periodik tanpa merusak kualitas piksel gambar seutuhnya.

Contoh Implementasi (Image Smoothing):

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Membaca gambar dalam format Grayscale (hitam putih)
# Pastikan Anda memiliki file gambar bernama 'gambar_test.jpg' di
folder yang sama
image = cv2.imread('/content/Adolf-Hitler-1933.webp',
cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Gambar tidak ditemukan! Pastikan path/nama file
benar.")
else:
    # ========================================
    # TAHAP 1: Transformasi ke Domain Frekuensi
    # ========================================
    # Melakukan Fast Fourier Transform 2D
    f_transform = np.fft.fft2(image)

    # Menggeser frekuensi nol (DC component) ke tengah matriks
    # Jika tidak digeser, frekuensi rendah akan berada di pojok-
pojok matriks
    f_shift = np.fft.fftshift(f_transform)
```

```
    # Menghitung magnitude spectrum hanya untuk visualisasi plot
(opsional)
    magnitude_spectrum = 20 * np.log(np.abs(f_shift) + 1)

    # ==========================================
    # TAHAP 2: Membuat & Mengaplikasikan Filter
    # ==========================================
    # Mendapatkan ukuran dimensi gambar
    rows, cols = image.shape
    crow, ccol = rows // 2, cols // 2  # Mencari titik tengah
matriks

    # Membuat mask (matriks penyaring) Ideal Lowpass Filter
    # Diisi dengan 0 (hitam/diblokir), lalu kita buat lingkaran 1
(putih/dilewatkan) di tengah
    mask = np.zeros((rows, cols), np.uint8)
    r = 20  # Radius lingkaran (semakin kecil radius, semakin
blur gambarnya)

    # Menggambar lingkaran putih (nilai 1) di tengah mask
    center = [crow, ccol]
    x, y = np.ogrid[:rows, :cols]
    mask_area = (x - center[0])**2 + (y - center[1])**2 <= r*r
    mask[mask_area] = 1

    # Mengalikan spektrum frekuensi gambar dengan mask (filtering
element-wise)
    f_shift_filtered = f_shift * mask

    # ==========================================
    # TAHAP 3: Inverse Transform (Kembali ke Gambar Asli)
    # ==========================================
    # Menggeser kembali matriks frekuensi dari tengah ke pojok
    f_ishift = np.fft.ifftshift(f_shift_filtered)

    # Melakukan Inverse Fast Fourier Transform 2D
    img_back = np.fft.ifft2(f_ishift)

    # Mengambil nilai absolut/nyata dari bilangan kompleks hasil
inversi
    img_back = np.abs(img_back)

    # ==========================================
    # Visualisasi Hasil
    # ==========================================
    plt.figure(figsize=(15, 5))

    plt.subplot(131)
```

```
    plt.imshow(image, cmap='gray')
    plt.title('Gambar Asli (Spasial)'), plt.axis('off')

    plt.subplot(132)
    # Visualisasi mask filter (opsional, agar terlihat area yang
dilewatkan)
    plt.imshow(mask, cmap='gray')
    plt.title(f'Ideal Lowpass Filter (Radius={r})'),
plt.axis('off')

    plt.subplot(133)
    plt.imshow(img_back, cmap='gray')
    plt.title('Hasil Image Smoothing'), plt.axis('off')

    plt.tight_layout()
    plt.show()
```



Pada tahap awal, citra hitam putih (domain spasial) dikonversi menjadi representasi frekuensi menggunakan fungsi np.fft.fft2. Hasil dari operasi matematis ini adalah matriks bilangan kompleks yang menyimpan komposisi frekuensi gambar tersebut. Namun, secara bawaan, algoritma FFT meletakkan komponen frekuensi paling rendah (frekuensi nol atau *DC component*, yang mewakili area warna mulus pada gambar) di keempat sudut matriks. Agar filter lebih mudah dirancang, fungsi np.fft.fftshift digunakan untuk menggeser susunan tersebut sehingga seluruh komponen frekuensi rendah berkumpul tepat di titik tengah matriks.

Setelah frekuensi terpusat, langkah selanjutnya adalah membangun instrumen *Ideal Lowpass Filter*. Kode Anda mencari titik tengah matriks, lalu menyiapkan sebuah matriks kosong berisi angka nol (merepresentasikan warna hitam atau pemblokiran) dengan ukuran yang sama persis seperti gambar asli. Di dalam matriks kosong tersebut, didefinisikan sebuah area lingkaran dengan radius 20 piksel tepat di titik tengah, yang seluruh nilainya diubah menjadi angka 1 (merepresentasikan warna putih atau pelolosan). Penyaringan gambar (*filtering*) secara praktis terjadi pada baris f_shift *

mask, di mana spektrum frekuensi dikalikan dengan topeng filter ini. Hasilnya, frekuensi rendah di dalam radius lingkaran dipertahankan (karena dikali 1), sedangkan frekuensi tinggi pembentuk detail tajam di luar lingkaran dihanguskan (karena dikali 0).

Matriks frekuensi yang telah dipangkas tersebut belum bisa dilihat sebagai gambar biasa, sehingga harus dikonversi balik. Pertama, susunan frekuensi yang berada di tengah dikembalikan lagi ke posisi asalnya di sudut-sudut matriks menggunakan np.fft.ifftshift. Setelah format susunannya sesuai, fungsi Inverse FFT (np.fft.ifft2) dijalankan untuk memulihkan matriks frekuensi tersebut menjadi susunan piksel spasial. Karena perhitungan IFFT masih menyisakan nilai bilangan kompleks, fungsi np.abs() digunakan untuk mengambil nilai magnitudo nyatanya saja. Hasil akhirnya adalah matriks riil yang siap ditampilkan sebagai gambar visual baru yang telah kehilangan detail kasarnya, sehingga tampak halus atau *blur*.

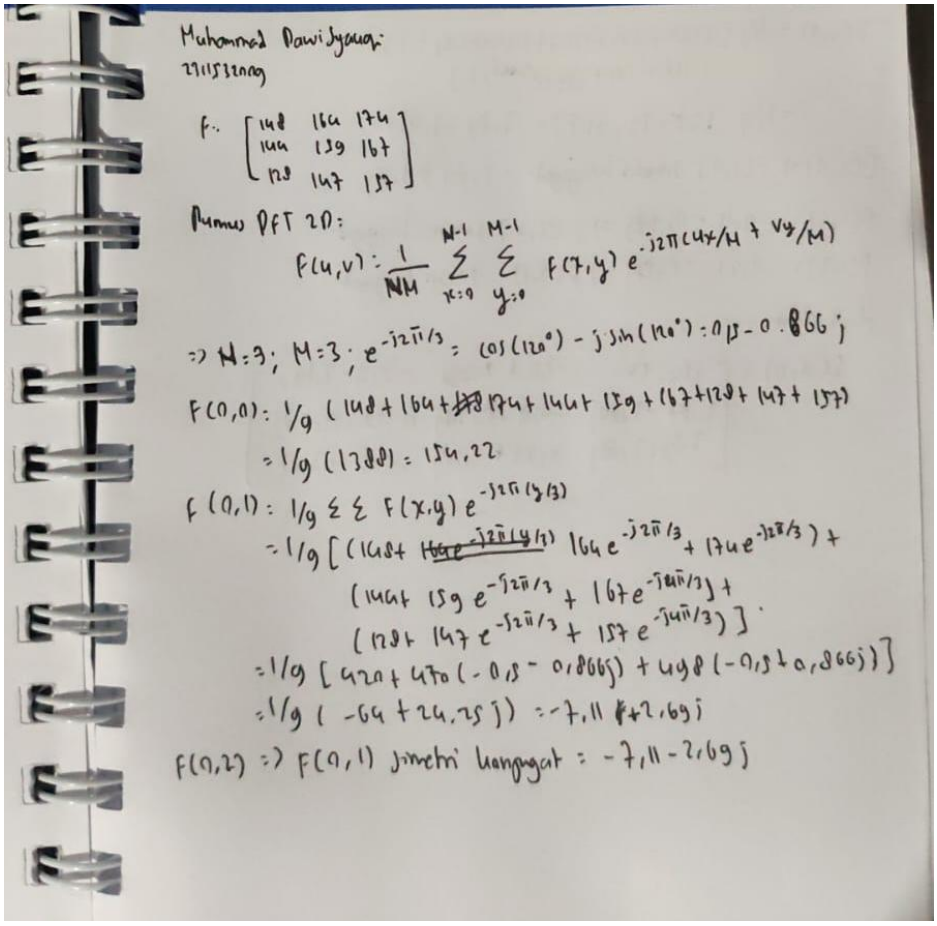
C. Link GitHub Tugas Pertemuan 6:

https://github.com/DawiSyauqi/2311532009_MuhammadDawiSyauqi_ImageProcessing/blob/main/Tugas_Fourier_Transform_Muhammad_Dawi_Syauqi.ipynb

D. Tugas

$$
\begin{array}{c|ccc}
(0,0) & & y & \\
\hline
 & 148 & 164 & 174 \\
x & 144 & 159 & 167 \\
 & 128 & 147 & 157
\end{array}
$$

Lakukan Fourier Transformation untuk Citra di atas



Muhammad Dawi Syauqi
2311532009

$$f = \begin{bmatrix} 148 & 164 & 174 \\ 144 & 159 & 167 \\ 128 & 147 & 157 \end{bmatrix}$$

Rumus DFT 2D:

$$F(u,v) = \frac{1}{NM} \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} f(x,y)\, e^{-j2\pi(ux/N + vy/M)}$$

$\Rightarrow N = 3;\ M = 3;\ e^{-j2\pi/3} = \cos(120^\circ) - j\sin(120^\circ) = 0{,}5 - 0{,}866j$

$F(0,0) = 1/9\,(148 + 164 + 174 + 144 + 159 + 167 + 128 + 147 + 157)$

$= 1/9\,(1388) = 154{,}22$

$F(0,1) = 1/9 \sum \sum f(x,y)\, e^{-j2\pi(y/3)}$

$= 1/9\,[(148 + 164\, e^{-j2\pi/3} + 174\, e^{-j2\pi/3}) +$

$(144 + 159\, e^{-j2\pi/3} + 167\, e^{-j4\pi/3}) +$

$(128 + 147\, e^{-j2\pi/3} + 157\, e^{-j4\pi/3})]$

$= 1/9\,[420 + 470(-0{,}5 - 0{,}866j) + 449(-0{,}5 + 0{,}866j)]$

$= 1/9\,(-64 + 24{,}25j) = -7{,}11 + 2{,}69j$

$F(0,2) \Rightarrow F(0,1)$ simetri konjugat $= -7{,}11 - 2{,}69j$

$$F(1,0) = \frac{1}{9}\left[(148 + 164 + 174) + (144 + 159 + 167)e^{-j2\pi/3} + (128 + 147 + 157)e^{-j4\pi/3}\right]$$

$$= \frac{1}{9}(35 - 32{,}91j) = 3{,}89 - 3{,}66j$$

$F(2,0) \Rightarrow F(1,0)$ simetri konjugat $= 3{,}89 + 3{,}66j$

$F(1,1) = -0{,}11 - 0{,}38j \Rightarrow F(2,2)$ simetri konjugat

$F(1,2) = 0{,}22 - 0{,}58j \Rightarrow F(2,1)$ simetri konjugat

Hasil akhir:

$$F(u,v) = \begin{bmatrix} 154{,}22 & -7{,}11 + 2{,}69j & -7{,}11 - 2{,}69j \\ 3{,}89 - 3{,}66j & -0{,}11 - 0{,}38j & 0{,}22 - 0{,}58j \\ 3{,}89 + 3{,}66j & 0{,}22 + 0{,}58j & -0{,}11 + 0{,}38j \end{bmatrix}$$